FraudGuard Enterprise

Real-Time UPI Fraud Detection & Explainable AI Architecture

Minor Project-I (AD-508) | SISTec Dept. of Artificial Intelligence & Data Science

Team: Wasil, Tayyab, Haseeb, Sufiyan

> 💡 **How to Use This Guide for Your Viva**
>
> This document is written in **plain English with zero confusing jargon**. Read it once with your team. Every question your teacher can randomly throw at you is already answered below with the exact winning explanation and real-life analogy.

1. What is FraudGuard in 30 Seconds? (The Elevator Pitch)

If your teacher says: "*In two sentences, what did you actually build?*"

> 🎯 **Your Exact Answer:**
>
> **"Ma'am, FraudGuard is a real-time fraud detection engine for digital banking and UPI transactions. When a transaction happens, our system extracts ledger telemetry, checks for balance discrepancies using our engineered equations, runs an XGBoost machine learning model in under 2 milliseconds, and provides Explainable AI (SHAP) reasons so compliance officers know exactly why a transaction was flagged or approved."**

Why is this important? Traditional banks use static rules like "*Flag any transfer greater than ₹50,000*". Fraudsters know this rule, so they transfer ₹49,999 twice! Our machine learning model doesn't just look at the amount; it analyzes **account drainage patterns** and mathematical inconsistencies that fraudsters cannot hide.

2. End-to-End System Architecture (How the Whole App Works)

There are 5 connected layers in our project. You can explain this step-by-step:

Layer	Technology Used	Plain-English Role
1. Frontend	HTML5, CSS3, Bootstrap 5.3, Chart.js	Clean FinTech dashboard where bank analysts test transactions on a simulator, view risk scores on an animated gauge, and review fraud alerts.
2. API Gateway	Django 4.2 LTS & Django REST Framework	Receives the transaction payload (Sender ID, Receiver ID, Amount, Balances) via `POST /api/transactions/`, validates numbers, and returns JSON.
3. Feature Engine	Python & NumPy	Computes the two critical engineered features (`errorBalanceOrig` and `errorBalanceDest`) in microseconds before feeding data to the ML model.
4. ML Engine	XGBoost 2.0.3 & TreeSHAP	Evaluates the decision trees, outputs a continuous probability between 0 and 1 (converted to a 0–100 Risk Score), and extracts feature attribution.
5. Database	MySQL 8.0 (InnoDB)	Stores transactions, high-risk fraud alerts, historical evaluation metrics, and dynamic threshold settings.

3. The Dataset: Why "PaySim" and Why Not Real UPI Bank Data?

> ⚠️ **Teacher Trap Question: "Why didn't you collect real Google Pay / PhonePe UPI transaction data?"**
>
> **Exact Answer:** *"Ma'am, real banking transaction logs contain customer PII (names, mobile numbers, bank balances, and UPI PIN hashes). Under Reserve Bank of India (RBI) regulations, the Banking Secrecy Act, and India's Digital Personal Data Protection (DPDP) Act, no bank or UPI payment app is legally allowed to share real customer transaction logs with students or the public."*

What is PaySim?

- PaySim is the **internationally recognized gold-standard academic dataset** for mobile money fraud research, created by financial researchers at NTNU Norway and published on Kaggle.
- It contains **6.36 million transactions** based on one month of real mobile money logs from an African cellular financial service, with simulated fraud attacks injected based on real fraud patterns.
- **Severe Class Imbalance:** Out of 6,362,620 transactions, only **8,213 are fraud (0.13%)** and **99.87% are normal**! This matches real-world banking where 999 out of 1,000 transactions are legitimate.
- **The 2 Risky Types:** PaySim has 5 transaction types (`PAYMENT` , `TRANSFER` , `CASH_OUT` , `DEBIT` , `CASH_IN`). Our data exploration revealed that **100% of frauds happen exclusively in `TRANSFER` and `CASH_OUT`** ! `PAYMENT` and `DEBIT` had ZERO fraud. Therefore, our model specifically filters and trains on `TRANSFER` and `CASH_OUT` .

4. Feature Engineering: The 2 Equations that Break Fraud

If you feed raw numbers (Amount, Balance) into an ML model, it gets confused. The real secret of our high performance is **Feature Engineering**.

In honest accounting, bank ledger math must add up:

```
Sender Old Balance - Amount = Sender New Balance
```

Therefore, we engineered two new balance discrepancy features:

> **Feature 1: Sender Balance Error (`errorBalanceOrig`)**
>
> `errorBalanceOrig = newbalanceOrig + amount - oldbalanceOrg`
>
> - **In Normal Transactions:** If you have ₹10,000 and send ₹2,000, your new balance is ₹8,000. `8,000 + 2,000 - 10,000 = 0` . The error is **ZERO**.
> - **In Fraud (Account Drain):** A hacker breaks into an account with ₹2,50,000. They transfer ₹2,50,000 away. The victim's new balance immediately drops to **₹0**! The mathematical error deviates wildly, instantly alerting the tree.

Feature 2: Receiver Balance Error (`errorBalanceDest`)

`errorBalanceDest = oldbalanceDest + amount - newbalanceDest`

- **In Normal Transactions:** Receiver balance before + amount = Receiver balance after. Error is **ZERO**.
- **In Fraud (Mule Accounts / Immediate Cash-out):** Fraudsters route stolen money through burner / mule accounts. Often, money is withdrawn at an ATM or agent cash-out booth at the exact same second, so the destination balance stays 0 or shows an impossible jump!

💡 **Teacher Question: "How do you know the receiver's balance before and after in real life?"**

Your Answer: *"Ma'am, FraudGuard is designed to sit at the **Bank Central Switch / Core Banking System (CBS)** or NPCI level, where both origin and destination account ledgers are visible. However, if the receiver is in an external bank and their live balance is private, our system automatically sets the destination balance to 0 and computes the error accordingly, relying on the sender drain discrepancy."*

5. The Machine Learning Model: XGBoost Explained Super Simply

If your teacher asks: "*What algorithm did you use, and how does it actually work?*"

- We used **XGBoost (Extreme Gradient Boosting)**.
- **Step 1 (Decision Tree):** Imagine a simple flowchart. "*Is Amount > ₹50,000? YES $\rightarrow$ Did sender balance become 0? YES $\rightarrow$ High Risk!*" A single tree is simple and can make mistakes.
- **Step 2 (Boosting):** Instead of one tree, XGBoost builds an **ensemble of 100+ trees sequentially** (one after another).
- **Step 3 (Gradient Learning):** Tree #1 makes predictions. It gets some transactions wrong. Tree #2 is specifically trained to **predict and fix the errors of Tree #1**. Tree #3 fixes the errors of Tree #2, and so on.

> ⚠️ **Teacher Trap Question: "Why didn't you use Deep Learning or Neural Networks (CNN/LSTM)?"**
>
> **Your Answer:**
> 1. **Tabular Data Superiority:** In academic benchmarks (like NeurIPS research), gradient boosted decision trees consistently outperform Deep Neural Networks on tabular/numerical financial data.
> 2. **Real-Time Latency:** In UPI, a payment must clear in **under 2 seconds**. XGBoost evaluates in **1.8 milliseconds on standard CPU**. Deep neural networks require expensive GPUs and take 50–200ms.
> 3. **Explainability:** Banking regulations require explainable decisions. XGBoost works seamlessly with TreeSHAP, whereas Deep Neural Networks are notorious black boxes.

6. Explainable AI (SHAP): Why Banks Hate Black Boxes

If your teacher asks: "*What is SHAP on your simulator and alert detail page?*"

- **SHAP stands for: SH**apley **A**dditive ex**P**lanations (based on Lloyd Shapley's Nobel Prize-winning Cooperative Game Theory).
- **The Problem:** If an AI blocks a customer's ₹50,000 transfer, the customer calls customer care crying: "*Why was my card blocked?*". If the bank says "*Because the AI said so*", the bank can be sued under RBI consumer protection rules.
- **The SHAP Solution:** SHAP assigns an **exact mathematical credit/blame score** to each input feature:
  - `errorBalanceOrig` contributed **+0.45** (Strongly raised fraud risk because account was drained).
  - `amount` contributed **+0.25** (High transfer amount raised risk).
  - `txn_type_TRANSFER` contributed **+0.10** (Transfer operation raised risk).
  - `newbalanceDest` contributed **-0.12** (Destination balance lowered risk slightly).
- **On the Website:** On our simulator, we color-code these in plain English: red badges for features that raised risk, and green badges for features that lowered risk!

7. Evaluation Metrics: Why "Accuracy" is a Trap!

> ⚠️ **The #1 Question Teachers Love to Ask: "What is your accuracy? Is 99% accuracy good?"**
>
> **Your Winning Answer:**
> *"Ma'am, in fraud detection, **Accuracy is a dangerous trap**. In our dataset, 99.87% of transactions are legitimate and only 0.13% are fraud. A completely useless 'dummy' model that approves 100% of all transactions will achieve **99.87% Accuracy**, yet it will catch **ZERO fraud attacks**! That is why accuracy is completely meaningless in imbalanced datasets."*

Instead of Accuracy, we evaluate on the 4 gold-standard metrics:

Metric	Our Score	Simple Meaning	Real-Life Analogy
Recall (Sensitivity)	99.57%	Of all the actual fraud attacks that happened, what percentage did our system catch?	**Airport Metal Detector:** It must catch 100% of weapons. Even if it beeps a few extra times on belts, NO weapon should get on the plane!
Precision	98.79%	When our system raises an alarm and says "*Fraud!*", how often is it actually right?	**False Accusations:** If police arrest 100 suspects, how many are actual thieves vs innocent citizens? High precision means innocent users are not harassed.
F1-Score	0.9918 (99.2%)	The harmonic mean that balances Precision and Recall together into one score.	A student who scores 99 in Math and 98 in Science has an F1 of ~98.5. It prevents cheating on one metric at the expense of the other.
PR-AUC	0.9984 (99.8%)	Area Under the Precision-Recall Curve across all possible thresholds.	The overall quality score of the model across both strict and lenient operating modes.

8. The Threshold Knob (Settings Page Explained)

If your teacher points to the **Settings Page Slider (1% to 99%)**:

- The ML model outputs a raw probability: e.g., $P(\text{fraud}) = 0.64$ (Risk Score = 64/100).
- **The Threshold is the decision boundary:**
  - If you **Lower the Threshold (e.g. 30%)**: The system becomes ultra-cautious. It catches 99.99% of fraud (High Recall), but it flags some innocent people (Lower Precision). Useful during high-risk festival shopping or cyber-attack waves.
  - If you **Raise the Threshold (e.g. 85%)**: The system only flags transactions it is 100% sure about. Zero false alarms for customers (High Precision), but sneaky small frauds might pass through (Lower Recall).
  - **Our Default: 50%** — The mathematically optimal balance point.

9. Top 10 Random Teacher Questions & Exact Answers

Memorize these 10 rapid-fire answers. You will sound like an industry professional:

Q1: Where is your model file stored and how does Django load it?
Answer: Our trained XGBoost model is serialized using `joblib` and saved at `ml/artifacts/model.joblib` along with `meta.json`. When Django starts, `transactions/model_loader.py` loads the model into server memory once, so it predicts instantly without reloading.

Q2: What is the response time / latency of your API?
Answer: The machine learning inference takes **1.8 milliseconds**. The full round-trip HTTP request (including Django JSON serialization and database check) takes approximately **15 to 25 milliseconds**, which is well within UPI's 2000ms SLA window.

Q3: What database are you using, and what tables exist?
Answer: We are using **MySQL 8.0** with InnoDB relational storage. We have 4 main tables: `transactions_transaction` (all logged transactions), `alerts_fraudalert` (flagged fraud with status new/reviewed/dismissed), `alerts_modelmetric` (evaluation history), and `core_risksetting` (active threshold knob).

Q4: How do you prevent SQL Injection and CSRF attacks in your project?
Answer: Django uses parameterized ORM queries, completely preventing raw SQL injection. For cross-site attacks, all forms and AJAX requests pass Django's `X-CSRFToken` header.

Q5: What happens if a brand-new zero-day fraud pattern appears tomorrow?
Answer: Supervised models like XGBoost learn historical patterns. For completely novel zero-day attacks, in production we would pair XGBoost with an unsupervised anomaly detector (like Isolation Forest) and retrain the XGBoost model weekly using the reviewed alerts from the database.

Q6: Why didn't you use Random Forest instead of XGBoost?
Answer: Random Forest builds trees in parallel using bagging. XGBoost builds trees sequentially using gradient boosting, correcting prior errors. In PaySim benchmarks, XGBoost achieved a higher PR-AUC (0.998 vs 0.985) with 4x faster prediction latency and smaller memory footprint.

Q7: What is the meaning of the 4 Preset Buttons on the Simulator?
Answer: They demonstrate real-world scenarios live:
1. *Account Drain:* Simulates an attacker stealing entire account balance.
2. *Drain + Cash-Out:* Simulates attacker draining funds to cash-out immediately.
3. *Normal UPI Transfer:* Routine ₹5,000 transfer with normal balances (returns 0 risk).
4. *Small Cash-Out:* Normal ₹2,000 ATM withdrawal (returns 0 risk).

Q8: What does the 'Mark as Reviewed' and 'Dismiss' button do on the alert page?
Answer: It provides Human-in-the-Loop (HITL) compliance workflow. A fraud analyst reviews the SHAP breakdown. If confirmed fraud, they mark it 'Reviewed' to freeze the account. If innocent customer, they mark it 'Dismissed', which updates MySQL via a `PATCH /api/alerts/<id>/` REST call.

Q9: How did you handle data splitting to prevent data leakage?
Answer: We split the dataset into 80% Training and 20% Testing using stratified splitting, ensuring the 0.13% fraud proportion was preserved in both sets, and calculated all scaling and feature parameters strictly on the training set.

Q10: Can this system run in Docker?
Answer: Yes, Ma'am. The project includes a production `Dockerfile` and `docker-compose.yml` configured with Gunicorn, WhiteNoise, and MySQL containerization.

10. Team Presentation Script & Role Delegation

Here is the recommended 5-minute presentation flow divided cleanly among team members:

Member 1 (Wasil - Project Lead & Intro): ~1 Minute

"Respected Ma'am/Sir, our project is FraudGuard — a Real-Time UPI & Banking Fraud Detection Engine. UPI processes billions of transactions, and static rule engines are failing against account takeover attacks. We built an intelligent system combining XGBoost machine learning, balance discrepancy feature engineering, and Explainable AI (SHAP) that evaluates payments in under 2 milliseconds. I will now hand over to Tayyab to explain the dataset and our ML model."

Member 2 (Tayyab - Data, Features & ML): ~1.5 Minutes

"Thank you, Wasil. Because real UPI logs are protected under RBI secrecy laws, we utilized the PaySim benchmark dataset containing 6.36 million mobile money transactions with severe 0.13% class imbalance. Rather than feeding raw amounts, we engineered two critical features: Sender Balance Error and Receiver Balance Error. When hackers drain an account, this ledger math deviates to zero. We trained an XGBoost gradient boosted classifier achieving 99.57% Recall and 98.79% Precision, verified with TreeSHAP local explainability. Haseeb will now demonstrate the live system."

Member 3 (Haseeb - Live Simulator & Alerts Demo): ~1.5 Minutes

"Thank you, Tayyab. Looking at our screen, this is our live FraudGuard console. On the Transaction Simulator, I will click 'Account Drain Attack'. Here, ₹2,50,000 is transferred, emptying the sender balance. When I run evaluation, in under 2ms our SVG gauge animates to 98% Risk, blocking the transaction. Below, SHAP reveals in plain English that 'Sender Balance Discrepancy' was the #1 culprit. Conversely, if I click 'Normal UPI Transfer', the gauge stays at 0% (Safe). This flagged event is immediately saved to MySQL and appears in our Fraud Alerts queue for analyst review. Sufiyan will now conclude with evaluation metrics."

Member 4 (Sufiyan - Metrics, Defense & Conclusion): ~1 Minute

"Thank you, Haseeb. On our Analytics dashboard, we track daily trends and model health. In fraud detection, standard Accuracy is a trap because predicting everything as normal gives 99.87% accuracy while missing all fraud. We therefore benchmarked on F1-Score (99.2%) and PR-AUC (99.8%). On our Settings page, bank compliance officers can adjust the sensitivity threshold knob to balance fraud recall against customer false alarms. In conclusion, FraudGuard delivers a secure, sub-2ms, explainable shield for real-time digital payments. We are now open for questions."